

Línea de producción con balanceo dinámico de carga

Un sistema distribuido que simula una planta conservera, detecta su cuello de botella en vivo y demuestra qué pasa cuando se escala la etapa lenta

César Olivares · Simón García-Huidobro

Evaluación 3 · Caso 2 · ECSD 2026-1 · Profesor: Daniel Calderón R. · 26 de agosto de 2026

PRIMER ACTO

El problema

Qué simula el sistema, y la pregunta que la simulación tiene que responder.

EL CASO

Cuatro etapas en serie, una replicada

Planta conservera ficticia de jurel en lata de 425 g. La unidad de trabajo es el lote.



- Entre etapa y etapa hay una **cola**: la fila de lotes esperando.
- Si a una etapa le llega trabajo más rápido de lo que procesa, su fila crece. Eso es un **cuello de botella**.

La pregunta del caso

¿Qué le pasa al cuello de botella cuando escalas la etapa lenta?

Adelanto: el cuello no desaparece — **se muda**. Todo lo que sigue es la evidencia de esa frase.

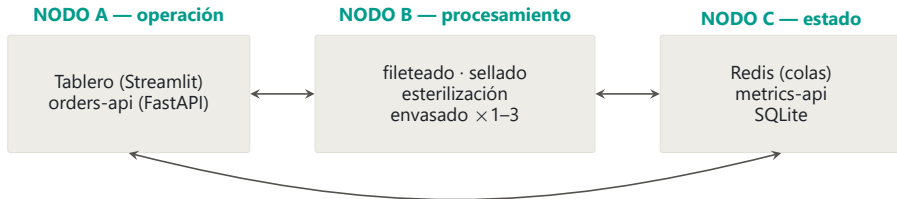
SEGUNDO ACTO

Cómo está construido

Cinco servicios, tres nodos, y una decisión de diseño que sostiene todo el resto.

Tres nodos y una sola imagen de estación

Cada nodo agrupa lo que escala por el mismo motivo. Las cuatro etapas corren el mismo programa.



- **B** se replica bajo carga; **C** guarda el estado compartido; **A** escala con los usuarios.
- Las cuatro estaciones son **el mismo código**, configurado por variables de entorno.

```
docker compose up -d  
--scale envasado=3
```

Separar nodos separa dominios de falla: si las estaciones saturan la CPU, el operador sigue atendido.

BALANCEO

Las réplicas piden trabajo; nadie se lo asigna

El enunciado pide «réplicas detrás de un balanceador». Nuestro balanceador es la cola compartida.

- Un round-robin es *estático*: le sigue mandando lotes a la réplica lenta o atascada.
- Con reparto por demanda, la réplica lenta **pide menos veces**.

La cola es el punto único

Por ella pasa todo el trabajo y ella decide quién procesa qué. Es lo que hace un balanceador, sin un proceso que lo haga.



La provocamos antes de resolverla

Las dos versiones están en el historial de commits: primero la rota, después la correcta.

Ingenua: mirar y sacar (2 pasos)



el mismo lote, dos veces

Atómica: BRPOP (1 operación)



0 duplicados en 200 órdenes

- Entre «mirar» (LRANGE) y «sacar» (LREM) hay una ventana en la que otra réplica ve la misma orden. La ensanchamos a propósito hasta reproducir duplicados.

Un lock con TTL **administra** esa ventana. La extracción atómica la **elimina**: es un mutex implícito por elemento, servido por el único hilo de comandos de Redis.

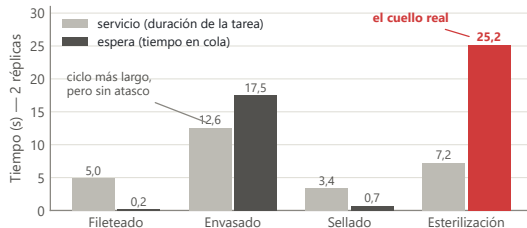
EL CRITERIO

Medimos espera, no servicio

Esta decisión es el corazón del proyecto: es lo que permite ver el cuello moverse.

- En el supermercado, un cajero lento no es un atasco. El atasco es **la fila que crece** frente a su caja.
- Cuello = etapa con mayor **espera promedio** en una ventana móvil de 60 s, contra umbrales relativos a su propio ciclo: advertencia $\geq 1,5\times$, crítico $\geq 3\times$.

Con 2 réplicas, envasado sigue teniendo el ciclo más largo (12 s) y **no** es el cuello. Midiendo servicio, el detector lo habría señalado para siempre.



INTERLUDIO

Demo en vivo

El sistema completo corriendo: tablero en localhost:8501, con lotes reales entrando a la línea.

DEMO

Qué vamos a mostrar, en orden

Levantado antes de empezar con `--scale envasado=2`. Aproximadamente cuatro minutos.

1. La línea al día: 4 etapas y 5 réplicas visibles, cada una con su ciclo y su carga.
2. Ingresamos lotes desde la interfaz, espaciados unos 5 s.
3. La espera de esterilización crece: el cartel pasa a **ámbar** y luego a **rojo**, diciendo por qué.
4. **La carrera, en vivo**: el botón «Ejecutar la comparación» corre las dos versiones del reclamo lado a lado — 300 envasados de 100 pedidos contra 100 de 100.
5. Matamos Redis: la interfaz **nombra al servicio caído** y el resto sigue vivo.

Es el mismo código

La comparación importa `servicios/comun/reclamo.py`, el módulo que usan las estaciones reales. No hay una copia para la demo que pueda quedar desincronizada.

1 comando

`docker compose up` — despliegue completo, sin pasos manuales

TERCER ACTO

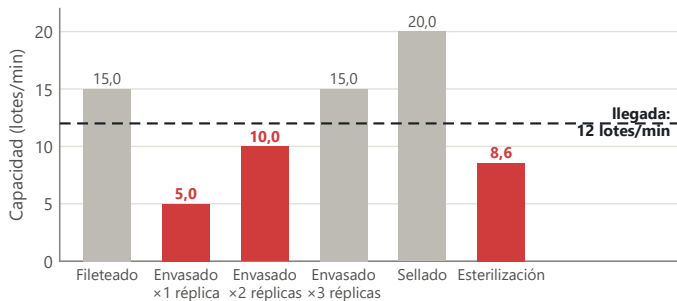
Lo que medimos

Cinco experimentos reproducibles. Ninguna afirmación de aquí en adelante es una opinión.

EL ESCENARIO

La aritmética antes del experimento

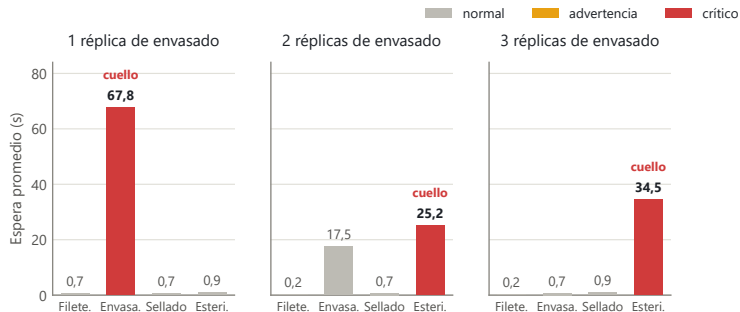
Inyección de una orden cada 5 s durante 180 s: 12 lotes por minuto entrando a la línea.



Toda etapa **bajo la línea de llegada** acumula fila tarde o temprano. Con 2 réplicas envasado sube a 10 lotes/min y esterilización (8,6) queda como el siguiente límite: la figura **anticipa** el resultado.

El cuello no se elimina: se muda

Espera promedio por etapa, según cuántas réplicas tenga envasado.

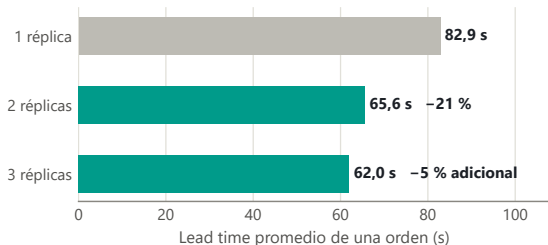


- Con 1 réplica, envasado acumula **67,8 s** de espera; el resto de la línea, casi ociosa.
- Con 2, el cuello **se muda a esterilización**. Con 3, envasado baja a 0,7 s... y el cuello no se mueve de ahí.

RENDIMIENTO

Cuánto compra realmente cada réplica

Lead time: lo que tarda un lote desde que se crea hasta que sale terminado.



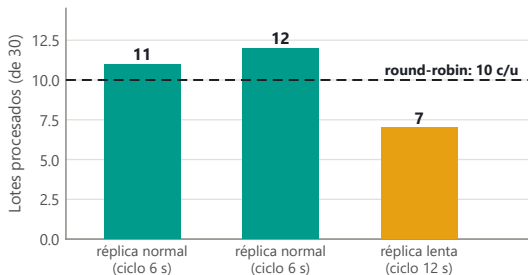
- La segunda réplica compra un **21 %** de mejora.
- La tercera, casi nada: el límite ya no está en envasado.

La siguiente inversión correcta sería **una segunda autoclave**, no una tercera envasadora. El tablero responde eso en vivo, que es para lo que sirve.

BALANCEO DINÁMICO

Qué pasa cuando una réplica es más lenta

Dos réplicas normales más una al doble de ciclo, sobre la misma cola, 30 lotes.



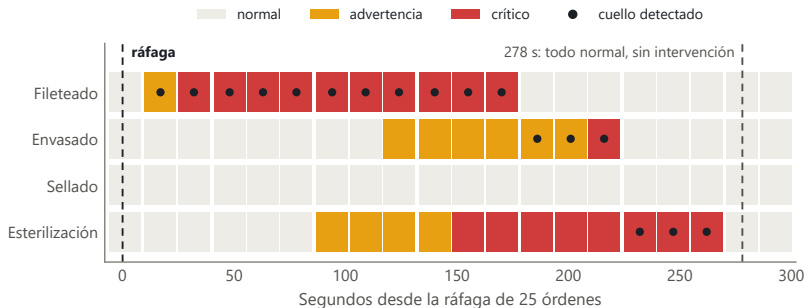
- Un round-robin habría repartido **10 / 10 / 10** sin mirar la velocidad de nadie.
- El reparto medido fue **11 / 12 / 7**.

Nadie programó ese reparto: salió de que cada réplica pide trabajo solo al quedar libre. Eso es balanceo *dinámico* — proporcional a la capacidad real.

SATURACIÓN

Se declara crítico solo, y se recupera solo

Ráfaga de 25 órdenes de golpe, sondeando /metricas/cuello cada 15 s.



A los **32 s** el sistema se declara crítico sin que nadie se lo diga; la congestión recorre la línea como una ola.

A los **278 s** todo vuelve a normal **sin intervención**: las esperas viejas salieron de la ventana móvil.

ROBUSTEZ

Errores, persistencia y despliegue

Lo que el enunciado exige como base, verificado contra el sistema corriendo.

2,1 s

respuesta 503 con plazo duro,
nombrando al servicio caído

El hallazgo: el cuelgue de 46 s con Redis
caído estaba en la resolución DNS, no
en el socket.

2 tablas

ordenes y eventos — 12 eventos por
orden completa

Espera, servicio y lead time **no se**
guardan: se calculan desde los eventos.
El dato guardado dos veces es el que se
contradice.

8/8

contenedores *healthy* en la prueba de
clon limpio

201 creación · 422 validación · 404
inexistente · 503 dependencia caída.

El healthcheck mide *vida*, no dependencias: un servicio que responde 503 explicándose está sano. La base sobrevive a `docker compose down`.

CUARTO ACTO

Lo que aprendimos

Los límites que conocemos, y las cinco frases que resumen el proyecto.

RESILIENCIA

No la exige el enunciado; sabemos dónde iría

La división en nodos deja los puntos de corte listos: cada mejora toca uno solo.

- **Estaciones** — sin estado propio: reconectan y mueren sin corromper nada. Pendiente: re-encolar el lote en curso si una réplica muere a mitad de ciclo (BLMOVE a una cola «en proceso» más un barrendero de latidos vencidos).
- **Redis** — punto único de falla del reparto: Redis Sentinel, o un broker con confirmaciones (ahí apuntaba el bonus de Kafka).
- **SQLite** — con más escritores o failover, migrar a MySQL cambiando solo `bd.py`.
- **APIs y tablero** — sin estado: dos instancias tras un balanceador HTTP quedan en alta disponibilidad, y los healthchecks que ya existen son su insumo.

Ninguna de estas cuatro mejoras obliga a rediseñar las otras tres. Para eso era la división en nodos.

CONCLUSIONES

Cinco frases, todas medidas

Cada una tiene su experimento reproducible en el informe y su script en el repositorio.

El cuello no se elimina: se muda

De envasado a esterilización al pasar de 1 a 2 réplicas. La tercera ya casi no compra nada.

Medir espera es lo que permite verlo

El cuello real tenía un ciclo más corto que envasado. Midiendo servicio no se habría encontrado nunca.

El balanceo dinámico emerge

11/12/7 con una réplica lenta, sin que nadie asignara ese reparto.

La carrera se elimina, no se administra

Extracción atómica: 0 duplicados en 200 órdenes, contra los duplicados sistemáticos de la versión ingenua.

Las fallas se explican, no se enmascaran

503 en 2,1 s nombrando al culpable real, en vez de 46 s de silencio.

Gracias

¿Preguntas?

César Olivares · Simón García-Huidobro · ECSD 2026-1 · USACH